

Algorithms and Data Structures for Data Science

Hashing and Exam 1 Review

CS 277

March 10, 2025

Brad Solomon



UNIVERSITY OF
ILLINOIS
URBANA - CHAMPAIGN

Department of Computer Science

Announcements

Fill out MP_generate feedback form for +2 EC (collectively)

Fill out IEF feedback form for +4 EC (collectively)

Exam 1 this week!

Friday will have a lab, it will be due the Thursday after Spring Break

Learning Objectives

Review the concepts behind hash tables

Foreshadow tradeoffs involved in hashing (motivate Sets)

A Hash Table based Dictionary

1	<code>d = {}</code>
2	<code>d[k] = v</code>
3	<code>print(d[k])</code>

A **Hash Table** consists of three things:

1. A hash function

2. A data storage structure

3. A method of addressing hash collisions

Hash Function

A hash function ***must*** be:

- **Deterministic:**
- **Efficient:**
- **Defined for a certain size table:**

General Hash Function

An $O(1)$ deterministic operation that maps all keys in a universe U to a defined range of integers $[0, \dots, m - 1]$

Deterministic:

A good hash function is one that has SUHA-properties:

Uniform:

Independent:

A Hash Table based Dictionary

1	<code>d = {}</code>
2	<code>d[k] = v</code>
3	<code>print(d[k])</code>

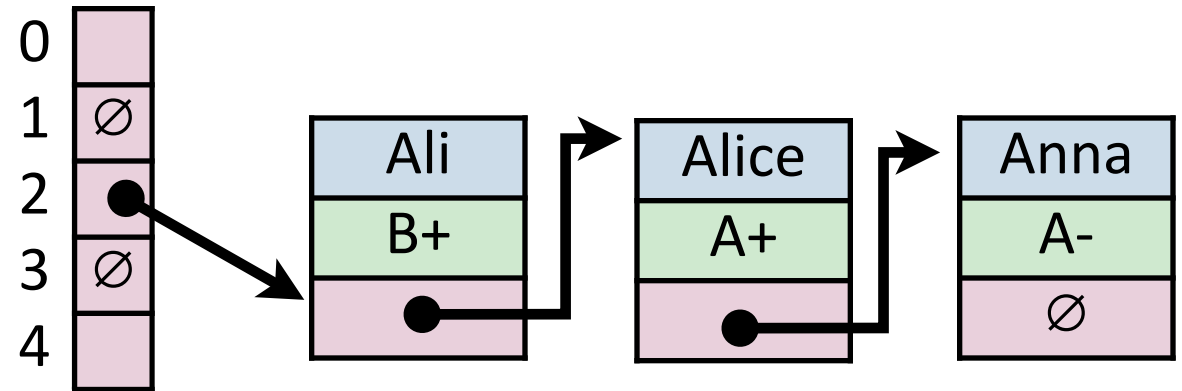
A **Hash Table** consists of three things:

1. A hash function
2. A data storage structure
3. A method of addressing hash collisions

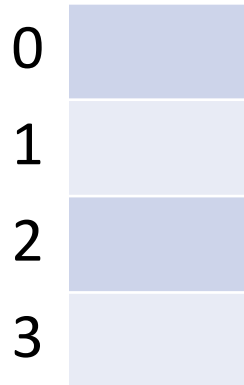
Open vs Closed Hashing

Addressing hash collisions depends on your storage structure.

- **Open Hashing:** store k, v pairs externally



- **Closed Hashing:** store k, v pairs in the hash table



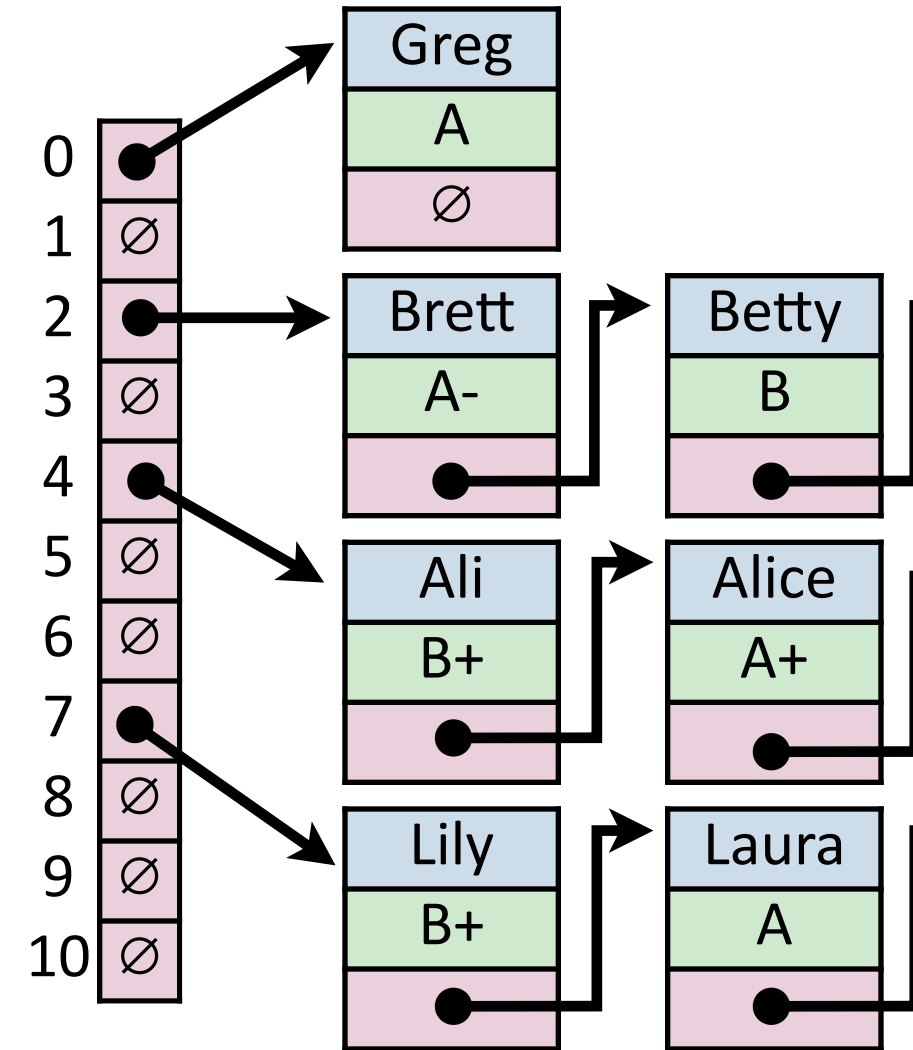
Collision Handling: Separate Chaining

For hash table of size m and n elements:

Find runs in: **$O(n)$**

Insert runs in: **$O(1)$**

Remove runs in: **$O(n)$**



Collision Handling: Probe-Based Hashing



A fixed array where a single (k, v) pair can exist in a cell

When a collision occurs, look at the **next available space**

When removing from the table, **tombstone** to preserve ability to find

When finding, continue searching until:

- Key is found

- A blank cell (with no tombstone) is found

- All positions have been searched

Collision Handling: Double Hashing

$S = \{ 16, 8, 4, 13, 29, 11, 22 \}$

$$h_1(k) = k \% 7$$

$$h_2(k) = 5 - (k \% 5)$$

$$|S| = n$$

$$|\text{Array}| = m$$

0	
1	8
2	16
3	
4	4
5	
6	13

$$h(k, i) = (h_1(k) + i * h_2(k)) \% 7$$

Try $h(k) = (k + 0 * h_2(k)) \% 7$, if full...

Try $h(k) = (k + 1 * h_2(k)) \% 7$, if full...

Try $h(k) = (k + 2 * h_2(k)) \% 7$, if full...

Try ...

Running Times

(Expectation under SUHA)

(Understand why we have these rough forms)

Open Hashing:

insert: _____.

find/ remove: _____.

Closed Hashing:

insert: _____.

find/ remove: _____.

Running Times (Expectation under SUHA)



Open Hashing: $0 \leq \alpha \leq \infty$

insert: $\frac{1}{1 - \alpha}$.

find/ remove: $\frac{1 + \alpha}{1 - \alpha}$.

Closed Hashing: $0 \leq \alpha < 1$

insert: $\frac{1}{1 - \alpha}$.

find/ remove: $\frac{1}{1 - \alpha}$.

Observe:

- As α increases:

- If α is constant:

Running Times

The expected number of probes for find(key) under SUHA

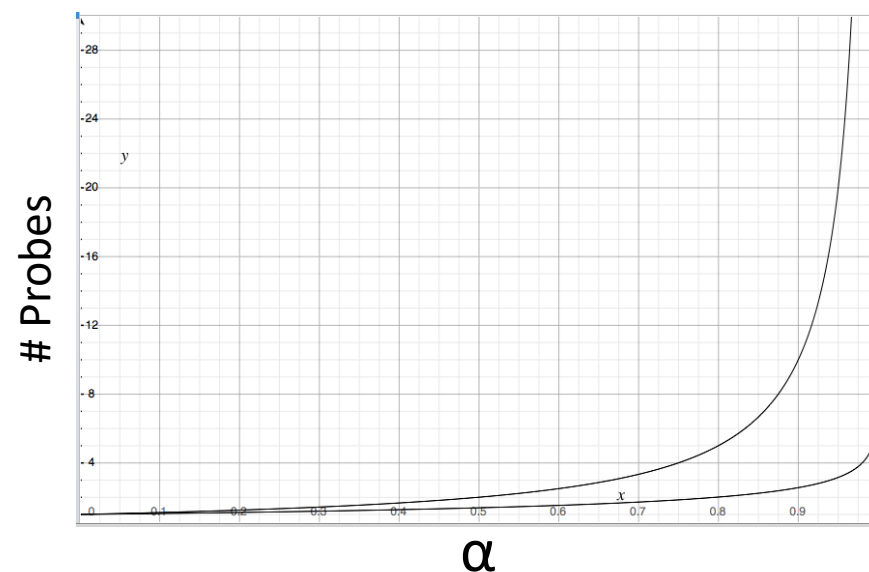
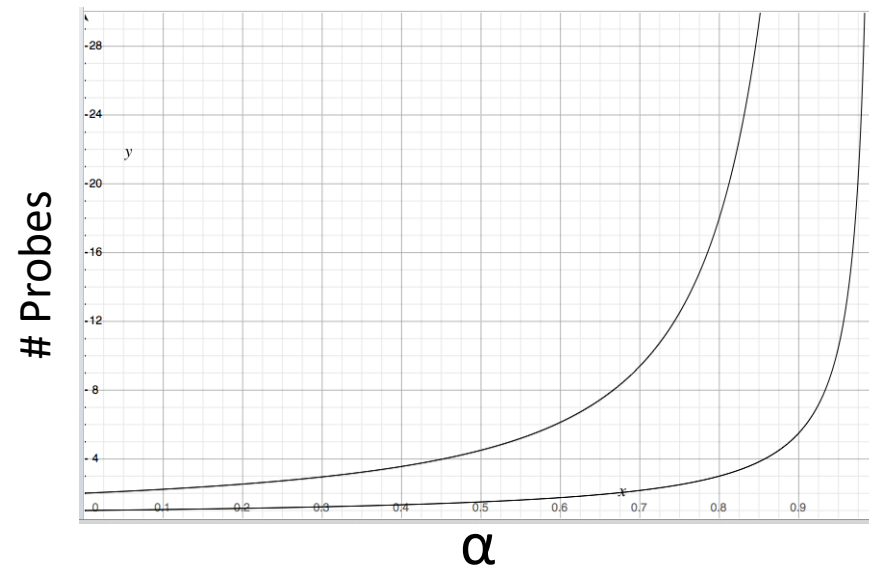
Linear Probing:

- Successful: $\frac{1}{2}(1 + 1/(1-\alpha))$
- Unsuccessful: $\frac{1}{2}(1 + 1/(1-\alpha))^2$

Double Hashing:

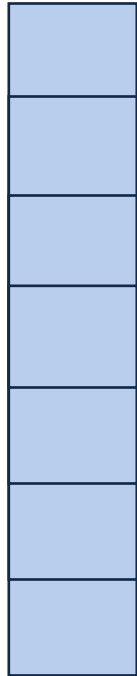
- Successful: $1/\alpha * \ln(1/(1-\alpha))$
- Unsuccessful: $1/(1-\alpha)$

When do we resize?



Resizing a hash table

How do you resize?



Running Times Review



	Hash Table	Array List	Linked List
Find	Expectation*:		
	Worst Case:		
Insert	Expectation*:		
	Worst Case:		
Remove	Expectation*:		
	Worst Case:		
Storage Space			

Thinking critically about hash tables: tradeoffs

Can we make our hash table better at some things? What is the cost?

Exam 1 Review

This is your time! (Not official class time)

What do you want to cover? What questions do you have?

List ADT

A list is an **ordered** collection of items

Items can be either **heterogeneous** or **homogenous**

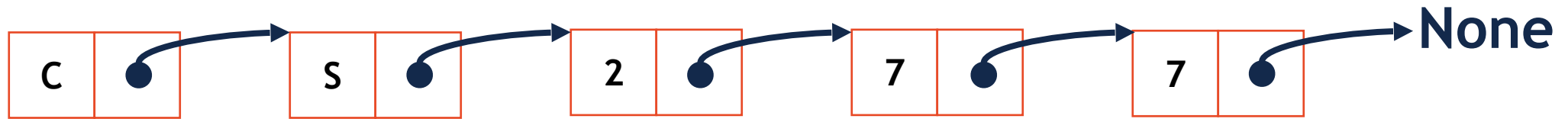
The list can be of a **fixed size** or is **resizable**

A minimal set of operations (that all lists should have):

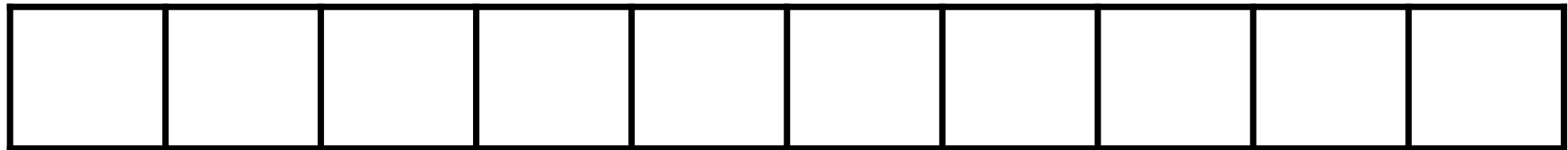
1. **Insert:** Add an item to a list
2. **Delete:** Remove an item from the list
3. **getData:** Look up the value at a specific index
4. **getSize:** Get the size (or length) of a list
5. **Constructor:** Create a new empty list

(Theoretical) List Implementations

1. Linked List



2. Array List



Python Class Definition

You should understand how to read and use a class object

```
1 class Circle:
2     pi = 3.14
3
4     def __init__(self, r, c, x, y):
5         self.radius = r
6         self.color = c
7         self.xpos, self.ypos = x, y
8
9     def __eq__(self, other):
10        return (self is other)
11
12    def circumference(self):
13        return 2 * Circle.pi * self.radius
14
15    def area(self):
16        return Circle.pi * (self.radius)**2
17
18
19
```

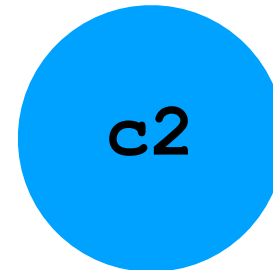
c1 = Circle(1, 0, 0, "Red")



c1.color

c1.area()

c2 = Circle(2, 0, 0, "Blue")

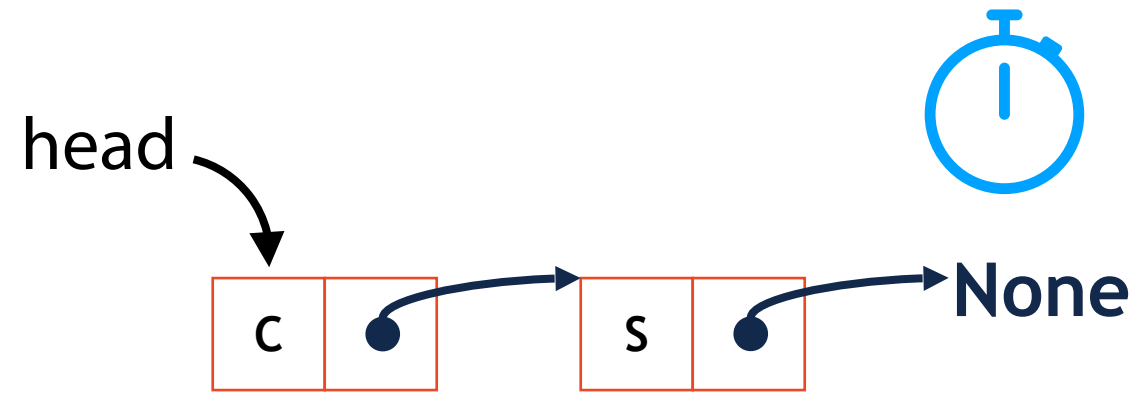


c2.color

c2.area()

Class Linked List

```
1 class Node:
2     def __init__(self, data, next=None):
3         self.data = data
4         self.next = next
5
6 class linkedList:
7     def __init__(self, head=None):
8         self.head = head
9
10    def __str__(self):
11
12    def __len__(self):
13
14    def __getitem__(self):
15
16    def add(self):
17
18    def insert(self):
19
20    def delete(self):
21
22    def remove(self):
23
```



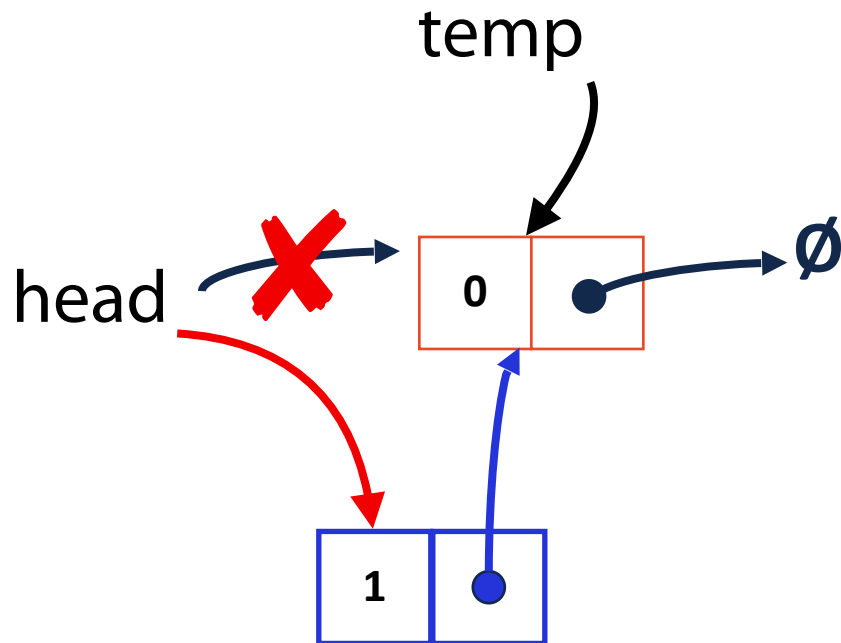
Linked List: add()

```
1 ll = linkedList()  
2  
3 for i in range(3):  
4     ll.add(i)
```

1. Make temp reference to head

2. Make new node (input=data, next=temp)

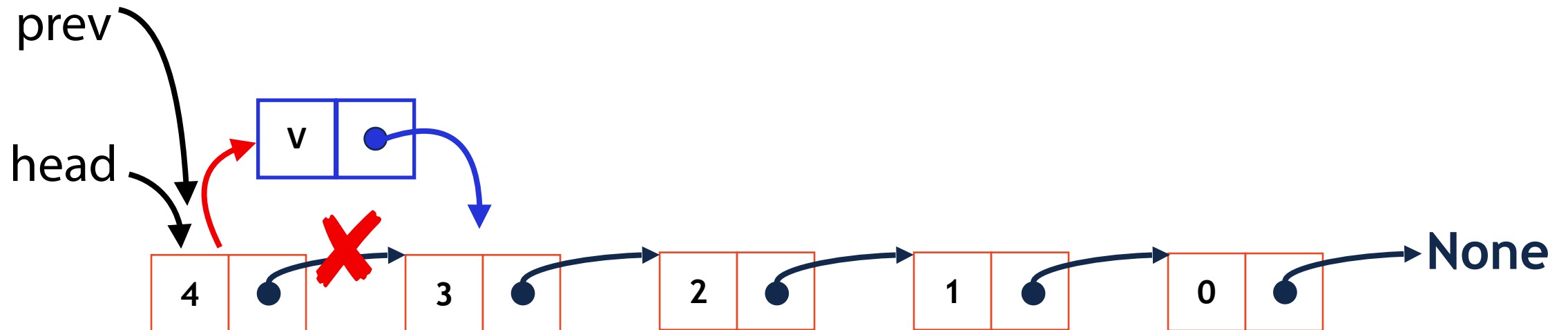
3. Update head to be new node



Linked List: insert()

```
1 for i in range(5):  
2     ll.add(i)  
3  
4 ll.insert("V", 1)  
5 print(ll)
```

1. Get previous* node to insert location
2. Make new node (data, next=prev.next)
3. Update prev.next to point to new node

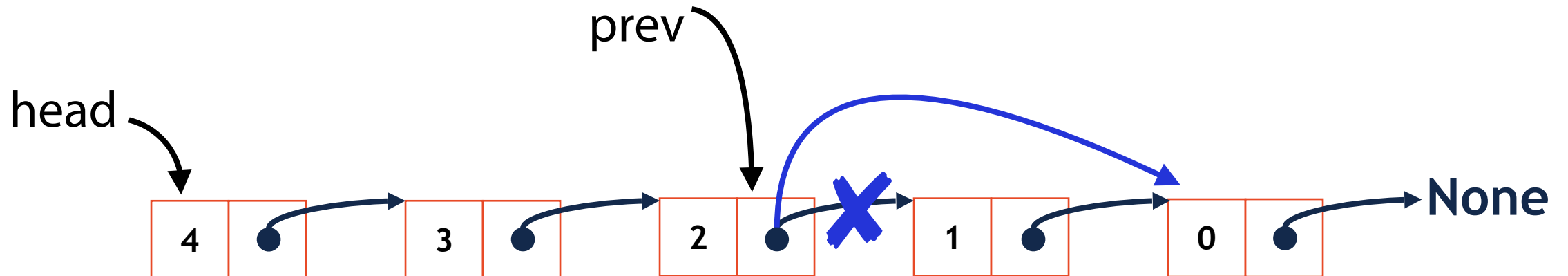


Linked List: delete()

```
1 for i in range(5):  
2     ll.add(i)  
3  
4 ll.delete(3)  
5 print(ll)
```

1. Get previous* node to insert location

2. Update prev.next to prev.next.next



List Abstract Data Type

A minimally functional list must have the following functions:

Constructor: `__init__()`

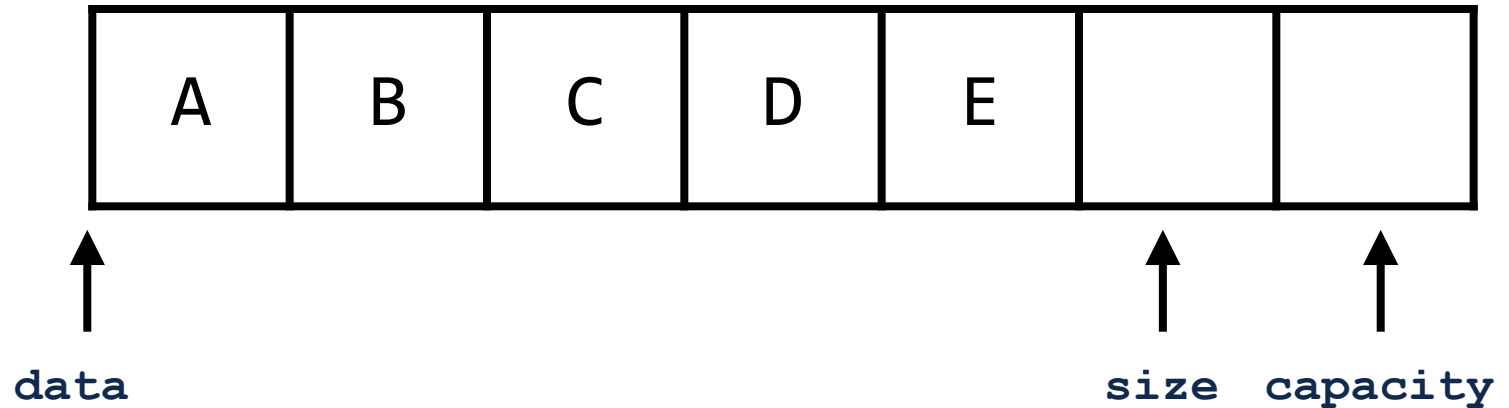
Insert: `append(x)` `insert(i, x)`

Delete: `remove(x)` `pop()`

getData: `[index]` `[start:stop:step]`

getSize: `len(list)`

Array List



Three values are necessary for efficient array usage:

- 1) **Data** — The start location of the array in memory
- 2) **Size** — How many objects are in our array
- 3) **Capacity** — The maximum space allocated in the array

List Implementation



	Singly Linked List	Array
Look up arbitrary location	$O(n)$	$O(1)$
Insert after given element	$O(1)$	$O(n)$
Remove after given element	$O(1)$	$O(n)$
Insert at arbitrary location	$O(n)$	$O(n)$
Remove at arbitrary location	$O(n)$	$O(n)$
Search for an input value	$O(n)$	$O(n)$

Special Cases:

Stack Data Structure

A minimally functional stack must have the following functions:

Constructor:

Push: $O(1)$ using linkedList **add()**

Pop: $O(1)$ using linkedList **removeFront()**

Peek: $O(1)$ using linkedList **head**

isEmpty(): $O(1)$ using linkedList **head**

Queue Data Structure

A minimally functional queue must have the following functions:

Constructor:

Enqueue: $O(1)$ if a **tail pointer** exists in linkedList

Dequeue: $O(1)$ using linkedList **removeFront()**

Front: $O(1)$ using linkedList **head**

isEmpty(): $O(1)$ using linkedList **head**

Know your labs / MPs!

MP_generate assessed your ability to use **random** and **file IO**

lab_fileIO tested your file IO

lab_matplot expanded your knowledge of matplotlib

lab_cipher tested your ability to parse and use lists in creative contexts

lab_quacks tested your ability to use the stack and queue